

OBJECT ORIENTED PROGRAMMING SYSTEM

OOPS :

Object Oriented Programming means writing a program by using **Classes** and **Objects** is known as OOPS.

➤ Features of OOPS :

There are 6 different features or Principles in OOPS

1. Class :

Class is the **Blueprint** or **Template** to create an **Object**.

Ex : class classname:
pass

2. Object :

Object is the **Instance** or **Copy** of a **Class**.

Ex : class classname:
Pass
Ob = classname()

Ex : # Student Details Example
class student:
def __init__(self,name):
self.name=name
def display(self):
print(self.name)
ob=student('Anil')
ob.display()

Pass :

- ✓ Pass is a Keyword used in Python with functions.
- ✓ It is used as **Null** Statement .
- ✓ It is used when no statements to be in function but function needed.

➤ Constructor :

- ❖ A Constructor is a **Special Method** present in a **Class**.
- ❖ It is **default method** in a **Class**.
- ❖ In Python constructor is defined using **__init__()** Method.
- ❖ It is automatically called when object is **Created**.
- ❖ **Constructors** are two types in Python. They are :
 1. Default Constructor or Non-Parameterized Constructor
 2. Parameterized Constructor.

➤ Self :

- ❖ Self is the **Default** Keyword Present in the Constructor.
- ❖ It **Points** to the **Current** object in the **Class**.
- ❖ It **holds** the Current Object **address** then it acts as Preroot for the Variables to access the Variables.

✓ **Default Constructor :**

- ✓ If a Constructor is Implemented **without using any Parameters** other than **Self** Keyword is known as Default Constructor.

Ex : # Default Constructor

```
class student:
    def __init__(self):
        print('Hello! I am Default Constructor')
ob=student()
```

✓ **Parameterized Constructor :**

- ✓ If a Constructor is Implemented by **using One or More Parameters Including Self** Keyword is known as Parameterized Constructor.

Ex : # Parameterized Constructor

```
class student:
    def __init__(self,name):
        self.name=name
        print('Hello! ',self.name)
        print('I am Parameterized Constructor')
ob=student('Anil')
```

3. Encapsulation :

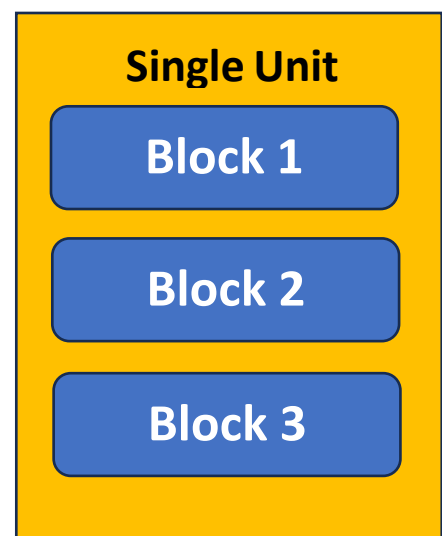
- **Encapsulation** means **Wrapping** up same type of **data** and **methods** in a **single unit**, and **restricting** direct **access** to data is called Encapsulation.
- For the **Restriction** of Data, **Access** Specifiers are Used like **Public, Private**, and **Protected**.

Ex : # Encapsualtion

```
class student:
    def __init__(self,name,rollno):
        self.name=name
        self.rollno=rollno

    def get_student(self):
        print('Student Name : ',self.name)
        print('Student RollNo : ',self.rollno)

ob=student('Anil',429)
ob.get_student()
```



- ✓ To **Restrict** the data **Access Specifiers** are **Used**.
- ✓ Access Specifiers are listed as **Public , Protected , Private**.

❖ Public :

A Public Member is accessible from anywhere Inside and Outside of class.

Ex : # Public

```
class student:
    def __init__(self):
        self.name='Anil'
s=student()
print(s.name)
```

❖ Protected :

- A Protected Member is Indicated by “_” (**Single Underscore**)
- Protected Members can be accessible within the class and subclasses Only.
- But ,In Python it can be same as Public Function but “_” is used before the Functions & Parameters.
- It helps the developers to Understand that the Methods & Variables are used with in the Classes & Subclasses Only.

Ex : # Protected

```
class student:
    def __init__(self,name):
        self._name='Anil'
s=student('Anil')
print(s._name)
```

❖ Private :

- A Private Member is Indicated by “__” (**Double Underscores**)
- Private Members & Methods can't be used outside of the Class.
- But, In Python there is another Possibility called **Name mangling** technique to access the Private members. **object._ClassName__variable**

Ex : # Private

```
class student:
    def __init__(self):
        self.__name='Anil'
s=student()
print(s.__name)
```

Output : Attribute error

Ex : # Private Using **Name Mangling** method

```
class student:
    def __init__(self):
        self.__name='Anil'
s=student()
print(s._student__name)
```

Output : Anil

➤ **Magic Methods (or) Dunder Methods :**

- ✓ Magic Methods are default methods present in Python. We can override them to customize how objects behave.
- ✓ Main Dunder Methods are :
- ✓ **Note :** “**Dunder**” Represents “**Double Underscore.**”

Magic Method	Purpose
<code>__init__()</code>	Constructor called when an object is created
<code>__str__()</code>	Returns a readable string representation of an object
<code>__repr__()</code>	Returns an official string representation of an object
<code>__len__()</code>	Supports the <code>len()</code> function
<code>__add__()</code>	Supports the <code>+</code> operator
<code>__sub__()</code>	Supports the <code>-</code> operator
<code>__mul__()</code>	Supports the <code>*</code> operator
<code>__truediv__()</code>	Supports the <code>/</code> operator
<code>__eq__()</code>	Supports the <code>==</code> operator
<code>__lt__()</code>	Supports the <code><</code> operator
<code>__gt__()</code>	Supports the <code>></code> operator
<code>__getitem__()</code>	Access items using indexing (<code>obj[index]</code>)
<code>__setitem__()</code>	Assign values using indexing (<code>obj[index] = value</code>)
<code>__call__()</code>	Makes an object callable like a function
<code>__del__()</code>	Destructor called when an object is destroyed

❖ Magic Method Examples :

1. __str__ Method :

```
class Student:
    def __init__(self):
        print('I am Student Class')
    def __str__(self):
        return 'Anil'
ob=Student()
print(ob)
```

Output : I am Student Class
Anil

2. __repr__ Method :

```
class Student:
    def __init__(self):
        print('I am Student Class')
    def __repr__(self):
        return 'Hello'
ob=Student()
print(ob)
```

Output : I am Student Class
Anil

3. __len__ Method :

```
class Student:
    def __init__(self,name):
        self.name=name
    def __len__(self):
        return len(self.name)
ob=Student('ANIL')
print(len(ob))
```

Output : 4

4. Math Methods :

```
class Student:
    def __init__(self,marks):
        self.marks=marks
    def __add__(self,other):
        return self.marks+other.marks
    def __sub__(self,other):
        return self.marks-other.marks
    def __mul__(self,other):
        return self.marks*other.marks
    def __truediv__(self,other):
        return self.marks/other.marks
```

```
ob1=Student(10)
ob2=Student(40)
print(ob1+ob2)
print(ob1-ob2)
print(ob1*ob2)
print(ob1/ob2)
```

Output : 50
-30
400
0.25

❖ Class Method :

- Class **Methods** are the methods that **belongs** to the class **Itself**.
- It is used to **access** the **Class Variables**.
- It is defined using “**@classmethod**” syntax.
- To **Access** the **Class Variables** we **don’t need** to **create** an **object**.

Ex 1 : # Class Method

```
class college:
    college_name='ABC'
    def __init__(self):
        pass

    @classmethod
    def get_college(cls):
        print(cls.college_name)

college.get_college()
```

Output : ABC

Ex 2 : class College:

```
    college_name='ABC College'
    l=[]
    def __init__(self,name,rollno):
        self.name=name
        self.rollno=rollno
        College.l.append([self.name,self.rollno])
    @classmethod
    def get_details(cls):
        print(cls.l)
ob=College('Anil',123)      # used with Object
ob=College('Sumanth',456)  # used with Object
College.get_details()      # used with Class
```

Output : [['Anil', 123], ['Sumanth', 456]]

❖ Static Method :

- In OOPS whenever you want to write a method without using Reference Pointers like Self or Cls we use Static Method.
- The method is must defined after a decorator **@staticmethod**.

Ex : # Static Method

```
class Student():
    def __init__(self,name):
        self.name=name
    @staticmethod
    def display():
        print('Student Method')

ob=Student('Anil')
ob.display()
```

Output : Student Method

4. Inheritance :

- Inheritance Mechanism is used to use One Class Methods and Parameters in an another class.
- It prevents from code duplication.
- Easy to maintain applications.
- Python Supports 5 types of Inheritances. They are :

1. Single Inheritance :

- When One Class inherits from One Parent Class is known as Single Inheritance.

Example :

Single Inheritance

```
class student:
```

```
    def __init__(self,name,rollno):  
        self.name=name  
        self.rollno=rollno
```

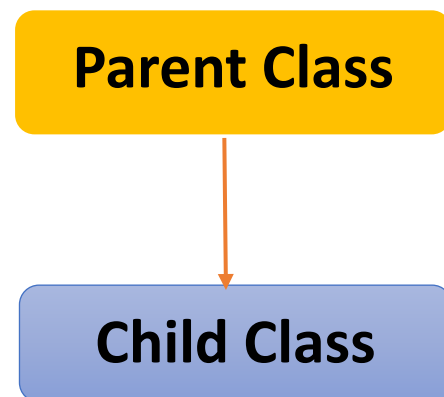
```
    def get_students(self):  
        print(self.name,self.rollno)
```

```
class record(student):
```

```
    def __init__(self,name,rollno,marks):  
        student.__init__(self,name,rollno)  
        self.marks=marks
```

```
ob=record('Anil',429,100)
```

```
ob.get_students()
```



Example 2 :

Single Inheritance

```
class student:
```

```
    def __init__(self,name,rollno):
```

```
        self.name=name
```

```
        self.rollno=rollno
```

```
    def get_students(self):
```

```
        print(self.name,self.rollno)
```

```
class record(student):
```

```
    def __init__(self,name,rollno,marks):
```

```
        super().__init__(name,rollno)
```

```
        self.marks=marks
```

```
ob=record('Anil',429,100)
```

```
ob.get_students()
```

Parent Constructor Invocation :

Calling Parent Class from Child Class

2. Multiple Inheritance :

- When One Class inherits from Multiple Parent Classes is known as Multiple Inheritance.

Example :

Multiple Inheritance

```
class Students:
```

```
    def display(self):
```

```
        print('I am as part of Students Class')
```

```
class Toppers:
```

```
    def show(self):
```

```
        print('I am as part of Topper Class')
```

```
class Students_10(Students,Toppers):
```

```
    def __init__(self):
```

```
        print('I am as part of Students_10 Class')
```

```
ob= Students_10()
```

```
ob.display()
```

```
ob.show()
```

Parent - 1

Parent - 2

Child

3. Multilevel Inheritance :

- When One Class inherits from another Class that already Inherited from another class.

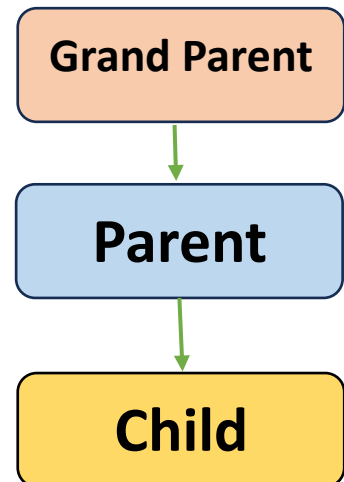
Ex : # Multilevel Inheritance

```
class Grandparent:
    def __init__(self):
        print('I am as part of Grandparent Class')
```

```
class Parent(Grandparent):
    def __init__(self):
        Grandparent.__init__(self)
        print('I am as part of Parent Class')
```

```
class Child(Parent):
    def __init__(self):
        Parent.__init__(self)
        print('I am as part of Child Class')
```

```
ob=Child()
```



4. Hierarchial Inheritance :

- If Multiple Child classes Inherited from a Single Parent class is known as Hierarchial Inheritance.

Ex : # Hierarchial Inheritance

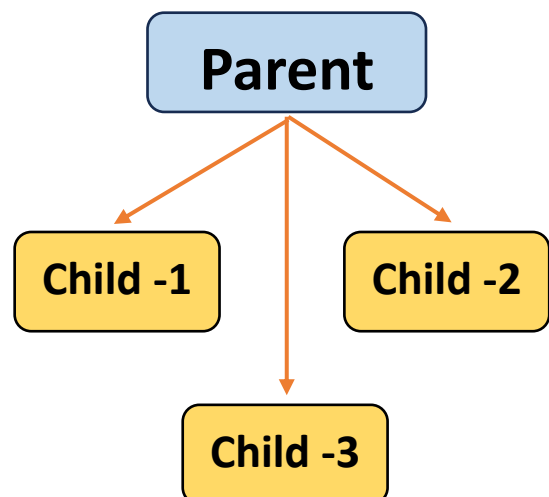
```
class Parent:
    def __init__(self):
        self.name="John-Doe"
```

```
class Son(Parent):
    def __init__(self):
        super().__init__()
        print(f'I am Son of {self.name}')
```

```
class Daughter(Parent):
    def __init__(self):
        super().__init__()
        print(f'I am Daughter of {self.name}')
```

```
ob=Son()
```

```
ob=Daughter()
```



Output :

I am Son of John-Doe

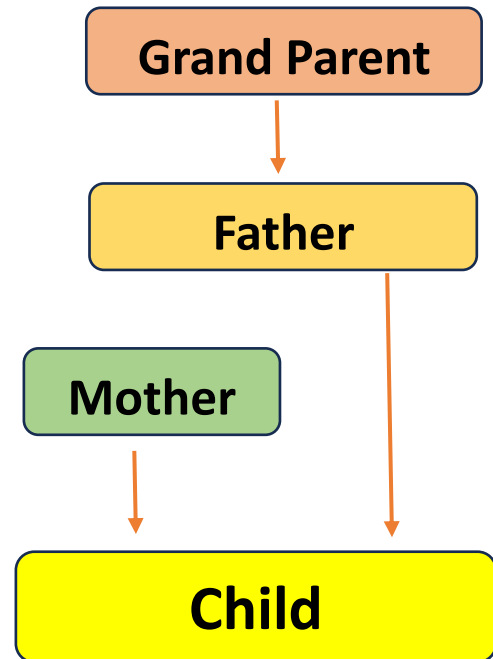
I am Daughter of John-Doe

5. Hybrid Inheritance :

- Combination of Multiple and Multilevel Inheritance is known as Hybrid Inheritance

Ex : **# Hybrid Inheritance**

```
class Grandfather:
    def display(self):
        print('I am Grandfather Class')
class Father(Grandfather):
    def show(self):
        print('I am Father Class')
class Mother:
    def get(self):
        print('I am Mother Class')
class Child(Father,Mother):
    def __init__(self):
        print('I am Child Class')
ob=Child()
ob.display()
ob.show()
ob.get()
```



Output :

I am Child Class
I am Grandfather Class
I am Father Class
I am Mother Class

Note : While executing inheritance, Python follows the Method Resolution Order (MRO) to search for methods.

Syntax : classname.mro()

✓ **Ex :** print(classname.mro())

5. Polymorphism :

- Polymorphism means **Many Forms**.
- Polymorphism means allowing an Object to take Many forms, allowing the same Method to behave differently based on Object.
- There are Two types in Polymorphism. They are:
 1. Compile Time Polymorphism (Method Overloading)
 2. Run Time Polymorphism (Method Overriding)

❖ Method Overloading :

- Redefining of a Method in a class with same name is Known as Method Overloading.
- Python actually takes the last defined method always.
- But in Java It is based on Requirement.
 - ✓ We can write two functions with same by differentiating with different Parameters.

Ex : # Method Overloading

```
class Student:
    def __init__(self,name,rn):
        self.name=name
        self.rn=rn
    def display(self):
        print('Student name is :',self.name)
    def display(self):
        print("Student name is :",self.name)
        print('His Roll Number is :',self.rn)
ob=Student('Anil',1234)
ob.display()
```

Output : Student name is : Anil
His Roll Number is : 1234

❖ Method Overriding :

- Redefining a Parent Class method in a Child Class with same **Name** and same **Parameters** is known as Method Overriding.
- It Replaces the Parent Class method implementation with Child Class method implementation.

Ex : # Method Overriding

```
class Parent:
    def __init__(self,name):
        self.name=name
    def display(self):
        print(self.name)
class child(Parent):
    def __init__(self,name,age):
        super().__init__(name)
        self.age=age
    def display(self):
        print(self.name,self.age)

ob=child('Anil',21)
ob.display()
```

Output : Anil 21

6. Abstraction :

- Abstraction means **Showing** the **Essential details** and **Hiding** the **Implementation details**.
- For Implementing abstraction we need to Import the ABC Class and abstract method from abc Module.
- Then Inherit ABC Class to the Class where you want to Implement Abstraction.
- Syntax : from abc import ABC,abstractmethod
class ClassName(ABC):
 @abstractmethod
 def method_name(self):
 pass

- If we Implement a class with abstract Method all the child class of it should have that abstract methods. Otherwise, It raise an error.

Example :

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self ):
        pass
class car(Vehicle):
    def start(self):
        print('Car Started')
class bike(Vehicle):
    def stop(self):
        print('Bike Stopped')
ob=car()
ob.start()
ob=bike()
ob.start()
```

Error

Because, All the Child Classes of Vehicle should have abstract method which is Start method in this Example.

Ex : # Abstract Method

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self ):
        pass
class car(Vehicle):
    def start(self):
        print('Car Started')
class bike(Vehicle):
    def start(self):
        print('Bike Started')

ob=car()
ob.start()
ob=bike()
ob.start()
```

Note :

Here, **Start** is the Abstract Method we just Informing the Child class to Implement the Abstract Method not giving the implementation that's why the definition is **Showing Necessary Data Hiding Implementation.**

Output : Car Started
Bike Started

7. Object Relationship :

- Object Relationship describe how Object Oriented Programming (OOP) related to each other.
- They are 3 types. They are :
 1. Association
 2. Aggregation
 3. Composition

❖ Association :

- Association is a type of Relationship where one Class can use another class Object. Both objects can exist Independently.

Ex : # Association

```
class Student:
    def __init__(self,name):
        self.name=name

class Teacher:
    def __init__(self,name):
        self.name=name
    def teaches(self,st):
        print(self.name,'Teaches',st.name)

st=Student('Anil')
teacher=Teacher('Kumar')
teacher.teaches(st)
```

Output : Kumar Teaches Anil

Explanation :

In this we will have two or more different classes and pass the one class object to another class method as Parameter there we use the Properties.

Note : Object address Received at Particular method.

❖ Aggregation :

- Aggregation is Weak “**Has-a**” Relationship where one Class contain Another Class Object. Both can Exist Independently.

➤ Ex : # Aggregation

```
class Student:
    def __init__(self,name):
        self.name=name

class Teacher:
    def __init__(self,name,st):
        self.name=name
        self.st=st
    def teaches(self):
        print(self.name,'Teaches',self.st.name)

st=Student('Anil')
teacher=Teacher('Kumar',st)
teacher.teaches()
```

Output : Kumar Teaches Anil

Explanation :

In this we will have two or more different classes and pass the one class object to another class as Parameter there we use the Properties.

Note : Object address Received at Constructor.

❖ Composition :

- Composition is Strong "**HAS-a**" Relationship where one Class called from another Class.
- The Child class can't Exist Independently.

Ex : # Composition

```
class Engine:
    def start(self):
        print('Car Started')

class Car:
    def __init__(self):
        self.Engine=Engine()
    def drive(self):
        self.Engine.start()

ob=Car()
ob.drive()
```

Output : Car Started

Explanation :

In this we will have two or more different classes one class called in another class there we use the Properties.